SOFTWARE REQUIREMENTS SPECIFICATION

(SRS)

Project Title

Smart Student Grievance and Complaint Management System

Technology Stack: Python · Flask · MongoDB · HTML · CSS · JavaScript

Web-Based Application

Submitted By	Details
Student Name	KAVINA M,DHESHINI S
Register Number	URK25CS1078,URK25CS1121
Department	CSE
Guide / Faculty	Mr.RISHAB HALDAR
Institution	karunya institute of technology and sciences
Academic Year	2026-2027

Table of Contents

1Introduction

2Overall Description

3System Features and Functional Requirements

4External Interface Requirements

5Non-Functional Requirements

6Hardware and Software Requirements

7System Design (Architecture and Database)

8Data Requirements

9System Constraints and Limitations

10Future Enhancements

11Conclusion

12References

1. Introduction

The Smart Student Grievance and Complaint Management System is a web-based application designed to help students submit complaints and grievances related to academic activities, hostel facilities, infrastructure, transportation, sanitation, and other college services. The system provides a centralized platform for recording, managing, tracking, and resolving complaints efficiently.

In traditional complaint-handling systems, students may need to submit written applications or approach staff members directly. These methods can lead to delays, missing records, and difficulties in tracking complaint status. The proposed system digitizes this process and improves communication between students and college administrators.

1.1 Purpose

The purpose of this Software Requirements Specification document is to define the functional and non-functional requirements of the Smart Student Grievance and Complaint Management System.

The objectives are:

- To provide a simple interface for students to register and log in.
- To enable students to submit complaints online.
- To allow students to track complaint status and view updates.
- To provide administrators with a centralized dashboard.
- To enable authorized administrators to assign, review, update, and resolve complaints.
- To maintain complaint history and student records securely.
- To improve transparency, accountability, and response time in complaint resolution.

This document serves as a reference for developers, testers, project guides, administrators, and other stakeholders involved in developing and evaluating the system.

1.2 Scope

The system will be developed as a web application accessible through a browser on a computer or mobile device.

The scope includes:

1. Student registration and authentication.
2. Secure login and logout.
3. Complaint submission with category, description, and optional attachments.
4. Automatic generation of a unique complaint ID.
5. Complaint tracking through status updates.
6. An administrator dashboard for viewing and managing complaints.
7. Complaint categorization and priority management.
8. Status updates and resolution remarks.
9. Student account management by authorized administrators.
10. Storage and retrieval of student and complaint records.

11. Complaint history for auditing and reference.

The initial version focuses on complaint submission, tracking, and administrative management. Advanced features such as AI-based complaint classification, sentiment analysis, automatic escalation, and mobile applications may be added in future versions.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
SGCM	Student Grievance and Complaint Management
UI	User Interface
DBMS	Database Management System
CRUD	Create, Read, Update, and Delete
Admin	Authorized system administrator
Authentication	Verification of a user's identity
Authorization	Determining what an authenticated user can access
Grievance	A concern or complaint raised by a student
Complaint ID	A unique identifier assigned to a complaint
Status	The current stage of a complaint
Frontend	The part of the application users interact with
Backend	The server-side logic that processes requests
API	Application Programming Interface
Flask	A Python web application framework
MongoDB	A document-oriented NoSQL database
HTTPS	Hypertext Transfer Protocol Secure

2. Overall Description

2.1 Product Perspective

The Smart Student Grievance and Complaint Management System is a standalone web-based information system that centralizes the college complaint-handling process.

The application follows a client-server architecture. Students and administrators interact with the frontend through a web browser. The backend processes requests, validates user input, enforces access permissions, and communicates with the database.

The major components are:

- **Student interface:** Registration, login, complaint submission, and complaint tracking.
- **Administrator interface:** Complaint review, prioritization, assignment, status updates, and user management.
- **Application server:** Processes business logic and handles requests.
- **Database:** Stores student information, administrator information, complaints, and complaint history.
- **Authentication and authorization module:** Controls access according to user roles.

The system can be extended to support email notifications, analytics, and automated complaint classification.

2.2 Product Functions

The main functions of the system are:

- Registering new student accounts.
- Authenticating registered users.
- Providing role-based access to students and administrators.
- Accepting new complaints and generating unique complaint IDs.
- Categorizing complaints by department or issue type.
- Displaying complaint details and current status.
- Allowing administrators to update complaint progress.
- Recording resolution comments and timestamps.
- Enabling authorized administrators to manage user accounts.
- Maintaining complaint records for reporting and auditing.

2.3 User Classes and Characteristics

The system supports the following user classes.

1. Student

Students are the primary users of the application. They should have basic computer or smartphone knowledge and be able to use a web browser.

Responsibilities:

- Register and log in.
- Submit accurate complaint details.

- View complaints they have submitted.
- Track complaint status.
- Read administrator remarks and resolution updates.

2. Administrator

Administrators are authorized college personnel responsible for managing complaints and student accounts.

Responsibilities:

- Log in using authorized credentials.
- View and filter submitted complaints.
- Assign complaint categories, priorities, or responsible departments.
- Update complaint status.
- Record remarks and resolution details.
- Manage student accounts and review complaint records.

3. System Maintainer

The system maintainer is responsible for technical operation and maintenance.

Responsibilities:

- Configure the application and database.
- Maintain backups and monitor errors.
- Apply software updates.
- Manage deployment and access controls.
- Investigate technical problems.

2.4 Operating Environment

The system is expected to operate in the following environment:

- **Client devices:** Desktop computers, laptops, tablets, and smartphones.
- **Operating systems:** Windows, Linux, macOS, Android, and iOS, subject to browser support.
- **Browsers:** Recent versions of Google Chrome, Microsoft Edge, Mozilla Firefox, or Safari.
- **Server environment:** A computer or hosting environment capable of running Python and Flask.
- **Database environment:** MongoDB running locally or on an authorized database server.
- **Network:** An internet or institutional network connection for accessing a remotely hosted application.
- **Development environment:** Visual Studio Code or another Python-compatible IDE.

2.5 Design and Implementation Constraints

The system must follow these constraints:

1. The application must use a browser-accessible interface.
2. The backend must be developed using Python and Flask, if the selected technology stack is retained.
3. MongoDB must be used for persistent data storage.
4. Only authenticated users may access protected features.
5. Students must not be able to view or modify another student's private complaints.

6. Administrative operations must be restricted to authorized roles.
7. Passwords must not be stored in plain text.
8. Input data must be validated before processing and storage.
9. The system must provide clear error messages without exposing confidential technical information.
10. The design must support future maintenance and feature expansion.

2.6 Assumptions and Dependencies

The system is based on the following assumptions:

- Students have access to a compatible device and browser.
- Users provide valid registration information.
- Administrators are authorized by the institution.
- The server and database are available during normal operation.
- Complaint records are retained according to institutional policies.
- Students understand that submitted complaints should contain accurate and relevant information.

Dependencies include Python, Flask, MongoDB, compatible web browsers, required Python packages, and the hosting environment. Email notifications, if implemented, will additionally depend on a configured email service.

3. System Features and Functional Requirements

Functional requirements define the operations that the system must perform to meet user needs. Each requirement is assigned an identifier for easier development and testing.

3.1 Student Registration

This module enables new students to create accounts in the system.

Description: Students must provide the required information through a registration form. The system validates the submitted details and creates an account if the information is acceptable.

Functional requirements:

- **FR-01:** The system shall display a student registration form.
- **FR-02:** The system shall collect the student's name, student ID, email address, department, year of study, and password.
- **FR-03:** The system shall validate mandatory fields and permitted input formats.
- **FR-04:** The system shall prevent duplicate registration using an existing student ID or email address.
- **FR-05:** The system shall securely hash the password before storing it.
- **FR-06:** The system shall display a confirmation message when registration is successful.
- **FR-07:** The system shall display an appropriate error message when registration fails.

Input: Student details and account credentials.
Output: Registration confirmation or validation error.
Precondition: The student does not already have an account.
Postcondition: A valid student account is stored in the database.

3.2 User Login and Authentication

This module verifies user credentials and provides access to authorized features.

Functional requirements:

- **FR-08:** The system shall provide a login form.
- **FR-09:** The system shall authenticate users using their registered credentials.
- **FR-10:** The system shall identify the user's role as student or administrator.
- **FR-11:** The system shall redirect authenticated users to the appropriate dashboard.
- **FR-12:** The system shall reject invalid login attempts with a suitable message.
- **FR-13:** The system shall maintain a secure authenticated session.
- **FR-14:** The system shall provide a logout function that terminates the session.
- **FR-15:** The system shall restrict access to protected pages based on authentication and role permissions.

Input: Username or registered email and password.
Output: Access to the appropriate dashboard or an authentication error.

3.3 Complaint Submission

This module allows students to report problems through an online form.

Functional requirements:

- **FR-16:** The system shall allow authenticated students to submit complaints.
- **FR-17:** The complaint form shall include a subject, category, description, and relevant details.
- **FR-18:** The system shall validate mandatory fields before accepting a complaint.
- **FR-19:** The system shall support optional file attachments where enabled.
- **FR-20:** The system shall generate a unique complaint ID for each successfully submitted complaint.
- **FR-21:** The system shall record the submission date and time.
- **FR-22:** The system shall associate each complaint with the submitting student's account.
- **FR-23:** The system shall store the complaint in the database.
- **FR-24:** The system shall display a submission confirmation and complaint ID.

Suggested complaint categories include academic issues, hostel facilities, transport, infrastructure, sanitation, library services, and other student concerns.

Input: Complaint subject, category, description, and optional attachment.
Output: A complaint record and its unique tracking ID.

3.4 Complaint Tracking

This module enables students to monitor the progress of their complaints.

Functional requirements:

- **FR-25:** The system shall display complaints submitted by the logged-in student.
- **FR-26:** The system shall allow students to search for a complaint using its complaint ID.
- **FR-27:** The system shall display the complaint subject, category, submission date, and current status.
- **FR-28:** The system shall display status updates and administrator remarks when available.
- **FR-29:** The system shall maintain a history of status changes.
- **FR-30:** The system shall prevent students from accessing complaints belonging to other students.

The complaint status may include:

- Submitted
- Under Review
- Assigned
- In Progress
- Resolved
- Rejected, with a reason where applicable
- Reopened, if the institution permits reopening

3.5 Admin Dashboard

The administrator dashboard provides a centralized view of complaint activity.

Functional requirements:

- **FR-31:** The system shall provide a dashboard accessible only to authorized administrators.
- **FR-32:** The dashboard shall display the total number of complaints.
- **FR-33:** The dashboard shall display counts of pending, in-progress, and resolved complaints.
- **FR-34:** The dashboard shall display a list of submitted complaints.
- **FR-35:** The system shall support filtering by category, status, priority, and submission date.
- **FR-36:** The system shall allow administrators to search complaints by complaint ID or student ID.
- **FR-37:** The dashboard shall provide access to complaint details and management actions.
- **FR-38:** The system shall display summary statistics to help administrators monitor complaint resolution.

Dashboard statistics must be calculated from actual stored records rather than fixed demonstration values.

3.6 Complaint Management and Status Updates

This module enables administrators to process complaints and record their progress.

Functional requirements:

- **FR-39:** The system shall allow authorized administrators to view complaint details.
- **FR-40:** The system shall allow administrators to update the complaint status.
- **FR-41:** The system shall allow administrators to assign complaints to the appropriate department or responsible staff member.
- **FR-42:** The system shall support complaint priority levels such as Low, Medium, High, and Urgent.
- **FR-43:** The system shall allow administrators to add remarks, actions taken, and resolution details.
- **FR-44:** The system shall record the administrator responsible for each update.
- **FR-45:** The system shall store the date and time of each status change.
- **FR-46:** The system shall preserve complaint history for accountability.
- **FR-47:** The system shall notify the student of important updates when a notification service is configured.
- **FR-48:** The system shall validate status transitions and restrict unauthorized changes.

A complaint should be marked as resolved only after the responsible administrator records the outcome. The system should retain the original complaint and its history instead of silently overwriting previous actions.

3.7 User Management

This module enables authorized administrators to manage student accounts.

Functional requirements:

- **FR-49:** The system shall allow administrators to view registered student accounts.
- **FR-50:** The system shall support searching for students using student ID, name, or email.
- **FR-51:** The system shall allow authorized administrators to activate or deactivate student accounts.
- **FR-52:** The system shall allow administrators to review basic account information.
- **FR-53:** The system shall prevent unauthorized users from changing account roles or privileges.
- **FR-54:** The system shall record significant administrative actions for auditing.

- **FR-55:** The system shall preserve complaint records associated with deactivated accounts according to retention policies.

Account deletion, if permitted, must follow institutional policies and applicable privacy requirements. Administrative access should not expose passwords or unnecessary confidential information.

Functional Requirements Summary

Module	Main requirement	Expected result
Registration	Validate and create accounts	Registered student
Authentication	Verify credentials and role	Authorized dashboard access
Complaint submission	Validate and save complaints	Unique complaint ID
Complaint tracking	Retrieve personal complaint records	Current status and history
Admin dashboard	Summarize and filter complaints	Centralized management view
Complaint management	Update status and resolution	Traceable complaint progress
User management	Manage accounts and permissions	Controlled user access

4. External Interface Requirements

External interface requirements describe how users and external software or hardware interact with the application.

4.1 User Interface Requirements

The user interface must be simple, consistent, responsive, and easy to navigate.

The student interface shall include:

- Registration page.
- Login page.
- Student dashboard.
- Complaint submission form.
- My Complaints page.
- Complaint details and tracking page.
- Profile and logout options.

The administrator interface shall include:

- Administrator login.
- Dashboard with complaint statistics.
- Complaint list and search tools.
- Complaint details and status update form.
- User management page.
- Administrative remarks and resolution fields.

Design guidelines:

1. Use clearly labeled input fields and buttons.
2. Display validation messages near the relevant fields.
3. Use consistent colors, fonts, spacing, and navigation.
4. Provide readable status labels and distinguishable priority indicators.
5. Design pages to adapt to desktop, tablet, and mobile screens.
6. Confirm important actions, such as resolving a complaint or deactivating an account.
7. Provide accessible labels, keyboard navigation, and sufficient color contrast.
8. Avoid displaying confidential student information unnecessarily.

4.2 Software Interface Requirements

The system shall communicate with the following software components.

Software component	Purpose
Web browser	Displays the application interface
HTML	Structures webpages
CSS	Controls layout and appearance

Software component	Purpose
JavaScript	Supports client-side interaction and validation
Python	Implements server-side logic
Flask	Handles web requests and application routes
MongoDB	Stores and retrieves system data
MongoDB driver	Connects the Flask application to MongoDB
Password-hashing library	Securely hashes and verifies passwords
Email service (optional)	Sends complaint notifications

The backend shall validate incoming requests before processing database operations. Database access shall be performed through authorized application code, not by exposing database credentials to the browser.

4.3 Hardware Interface Requirements

The application does not require specialized hardware.

Client-side hardware:

- Desktop computer or laptop.
- Smartphone or tablet, if supported by the browser.
- Keyboard, mouse, or touchscreen.
- Network interface for remote access.

Server-side hardware:

- A computer or virtual server capable of running Python and MongoDB.
- Sufficient memory and storage for application execution and complaint records.
- Network connectivity for remote access and optional notifications.
- Backup storage appropriate to the institution's retention policy.

No dedicated sensor, printer, or other specialized device is required for the core system.

4.4 Communication Interface Requirements

The system shall use standard web communication protocols.

- HTTP or HTTPS may be used during local development; HTTPS shall be used in production.
- Browser requests shall communicate with the Flask backend through web routes or APIs.
- The backend shall communicate with MongoDB through the configured database driver.
- The application shall use appropriate request methods, such as GET for retrieval and POST for submission, with suitable methods for updates and deletions.
- Session cookies shall be configured securely in production.
- Uploaded files, if supported, shall be validated for type and size and stored with appropriate access restrictions.
- Email notifications, if implemented, shall use a properly configured mail service.

Sensitive information must be protected in transit and must not be included unnecessarily in URLs or application logs.

5. Non-Functional Requirements

Non-functional requirements define the quality, security, performance, and operational characteristics expected from the system.

5.1 Performance Requirements

- **NFR-01:** Common pages should load within three seconds under normal test conditions.
- **NFR-02:** Complaint submission should complete within three seconds under normal test conditions, excluding delays caused by external services.
- **NFR-03:** Complaint search and status retrieval should return results within two seconds for the defined test dataset.
- **NFR-04:** The system should support multiple concurrent users according to the deployment capacity.
- **NFR-05:** Database queries should use appropriate indexes for frequently searched fields.
- **NFR-06:** Dashboard statistics should be generated efficiently from stored data.
- **NFR-07:** Large complaint lists should support pagination rather than loading all records at once.

These values are proposed targets for testing, not claims about measured performance. Final limits should be validated against the expected number of users and the deployment environment.

5.2 Security Requirements

- **NFR-08:** Passwords shall be stored using a suitable password-hashing algorithm and shall never be stored in plain text.
- **NFR-09:** The system shall enforce authentication for protected features.
- **NFR-10:** Role-based access control shall restrict student and administrator operations.
- **NFR-11:** Students shall only access their own complaint records unless an explicitly authorized workflow allows otherwise.
- **NFR-12:** All input shall be validated and handled safely to reduce injection, cross-site scripting, and related attacks.
- **NFR-13:** The production application shall use HTTPS.
- **NFR-14:** Session cookies shall use suitable security settings, including Secure, HttpOnly, and SameSite attributes as appropriate.
- **NFR-15:** Database credentials and secret keys shall be stored outside publicly accessible source code.
- **NFR-16:** Administrative operations and significant complaint changes shall be logged without recording passwords or unnecessary sensitive data.
- **NFR-17:** The system shall implement suitable login protections, such as rate limiting.
- **NFR-18:** Uploaded files shall be validated and protected against unauthorized access.
- **NFR-19:** Backups and retained complaint records shall be protected against unauthorized access.

5.3 Usability Requirements

- **NFR-20:** A student shall be able to submit a complaint without specialized technical knowledge.

- **NFR-21:** Navigation shall remain consistent across application pages.
- **NFR-22:** Form fields shall have meaningful labels and instructions.
- **NFR-23:** Success and error messages shall clearly explain the outcome of an operation.
- **NFR-24:** The interface shall be responsive across supported screen sizes.
- **NFR-25:** Complaint statuses shall use understandable text rather than relying only on colors or icons.
- **NFR-26:** The system shall minimize unnecessary steps in complaint submission and tracking.

5.4 Reliability Requirements

- **NFR-27:** The system shall preserve successfully stored complaint records.
- **NFR-28:** The system shall validate database operations and handle failures gracefully.
- **NFR-29:** The system shall prevent duplicate submissions where practical, including accidental repeated submissions.
- **NFR-30:** The system shall maintain complaint status history to support recovery and auditing.
- **NFR-31:** Database backups shall be created according to a defined schedule.
- **NFR-32:** The application shall provide clear error handling when the database or another required service becomes unavailable.
- **NFR-33:** Recovery procedures shall be documented and tested.

5.5 Maintainability Requirements

- **NFR-34:** The source code shall be organized into logical modules, such as routes, services, database access, templates, and configuration.
- **NFR-35:** Configuration values shall be separated from business logic.
- **NFR-36:** The code shall use meaningful variable names and appropriate documentation.
- **NFR-37:** Functional changes shall be testable without requiring a complete rewrite of the application.
- **NFR-38:** Database indexes and schema validation rules shall be documented.
- **NFR-39:** The project shall maintain dependency information and setup instructions.
- **NFR-40:** The application shall support future enhancements, including notification services and complaint analytics.

6. Hardware and Software Requirements

This section specifies the minimum development and deployment environment required to implement and operate the proposed system. The values below are suggested baseline requirements and may be adjusted according to workload.

6.1 Hardware Requirements

Component	Minimum suggested requirement
Processor	Dual-core processor, 2 GHz or equivalent
RAM	4 GB
Storage	5 GB of available space for development and initial data
Display	1366 × 768 resolution or higher
Input devices	Keyboard and mouse or touchscreen
Network	Internet or institutional network for remote access
Server	Computer or virtual server capable of running Python and MongoDB
Backup storage	Sufficient storage for database backups and retained records

A development machine may run the Flask application and MongoDB locally. A production deployment should be sized according to concurrent users, complaint volume, attachment storage, and backup requirements.

6.2 Software Requirements

Software	Purpose
Windows, Linux, or macOS	Operating system
Python 3	Backend programming language
Flask	Web application framework
MongoDB	Database management
MongoDB Python driver	Database connectivity
HTML5	Webpage structure
CSS3	Styling and responsive layout
JavaScript	Interactive frontend features
Visual Studio Code	Development environment
Google Chrome or equivalent	Testing and application access
Git	Version control, if used

6.3 Technology Stack

The following technology stack is proposed for the project.

Layer	Technology	Role
Frontend	HTML5, CSS3, JavaScript	Provides forms, dashboards, tables, navigation, and interactive features.
Backend	Python with Flask	Handles requests, validation, authentication, business rules, and complaint management.
Database	MongoDB	Stores student profiles, administrator accounts, complaints, and status history.
Development tools	Visual Studio Code and Git	Support coding, debugging, testing, and version control.

The selected stack separates the presentation layer, application logic, and data storage. This separation makes the system easier to test, maintain, and extend.

7. System Design

System design describes the major components, interactions, data movement, and database structure of the application.

7.1 System Architecture Diagram

The application follows a three-layer architecture consisting of the presentation layer, application layer, and data layer.

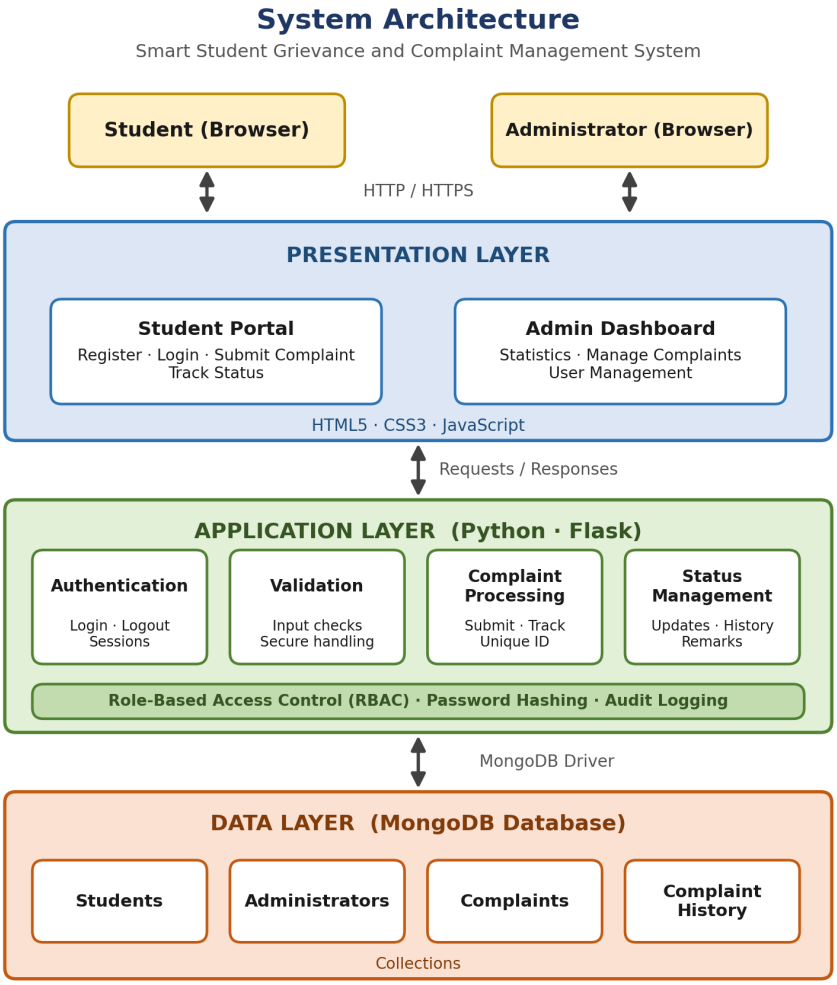


Figure 7.1: Proposed system architecture.

Architecture explanation:

1. The presentation layer displays pages and accepts user input.
2. The application layer validates requests, checks permissions, processes complaints, and manages status changes.
3. The data layer stores and retrieves records using the database driver.
4. The results are returned to the frontend and displayed to the user.

The browser communicates with the Flask server through HTTP or HTTPS. The server communicates with MongoDB through the configured database connection. The database must not be accessed directly by the browser.

7.2 Database Design

The database design defines how information is organized and connected. MongoDB stores information as documents within collections. The following collections are proposed.

1. Students Collection

Field	Data type	Description
_id	ObjectId	Unique database identifier
student_id	String	Institutional student ID
name	String	Student name
email	String	Registered email
department	String	Department name
year	Number	Year of study
password_hash	String	Securely hashed password
is_active	Boolean	Account status
created_at	Date	Registration timestamp

2. Administrators Collection

Field	Data type	Description
_id	ObjectId	Unique administrator identifier
name	String	Administrator name
email	String	Login email
password_hash	String	Securely hashed password
role	String	Administrator role
is_active	Boolean	Account status
created_at	Date	Account creation timestamp

3. Complaints Collection

Field	Data type	Description
_id	ObjectId	Database identifier
complaint_id	String	Unique complaint tracking ID
student_id	String or ObjectId	Reference to submitting student

Field	Data type	Description
subject	String	Complaint title
category	String	Complaint category
description	String	Complaint details
priority	String	Low, Medium, High, or Urgent
status	String	Current complaint status
assigned_to	String or ObjectId	Responsible staff member, if assigned
resolution_remarks	String	Resolution details
created_at	Date	Submission timestamp
updated_at	Date	Last update timestamp
resolved_at	Date or null	Resolution timestamp

4. Complaint History Collection

Field	Data type	Description
_id	ObjectId	Unique history record
complaint_id	String or ObjectId	Related complaint
old_status	String	Previous status
new_status	String	Updated status
remarks	String	Update explanation
updated_by	String or ObjectId	Administrator who made the change
updated_at	Date	Timestamp of change

Database relationships:

- One student may submit multiple complaints.
- Each complaint belongs to one student.
- One complaint may have multiple history records.
- An administrator may manage multiple complaints.
- Each history record references the complaint and the administrator responsible for the update, where applicable.

The database should enforce uniqueness for student IDs, registered emails, and complaint IDs. Indexes should be created for frequently queried fields such as student_id, complaint_id, status, and created_at.

For implementation, references should use consistent identifier types. Sensitive fields must be excluded from routine API responses. Database validation and access permissions should be configured to maintain data integrity.

8. Data Requirements

This section identifies the data collected, processed, stored, and retrieved by the system.

8.1 Student Information

The system shall maintain the information required to identify students and associate complaints with the correct accounts. The student information includes:

- Student ID
- Full name
- Registered email address
- Department
- Year of study
- Password hash
- Account status
- Account creation timestamp

The system shall validate student information before saving it. Student IDs and email addresses must be unique where required. Access to student information shall be limited to authorized operations.

8.2 Complaint Information

The system shall store the details required to process and track complaints. The complaint information includes:

- Unique complaint ID
- Student reference
- Complaint subject
- Complaint category
- Complaint description
- Optional supporting attachments
- Priority level
- Current status
- Assigned department or staff member
- Submission date and time
- Last update date and time
- Resolution remarks

The system shall ensure that each complaint is associated with the correct student and that mandatory fields are provided before submission.

8.3 Administrator Information

The system shall maintain administrator account information required for authorized access and complaint management. The information includes:

- Administrator ID
- Name
- Registered email address
- Password hash
- Role and permissions
- Account status
- Account creation timestamp

Administrator accounts shall be created or approved through a controlled process. Public student registration must not provide a way to create administrator accounts or grant administrator privileges.

8.4 Complaint Status and Records

The system shall maintain the current status and historical changes for every complaint. The status record shall include:

- Complaint ID
- Previous status
- New status
- Date and time of update
- Administrator or authorized staff member responsible
- Remarks explaining the action taken
- Resolution details, when applicable

The system shall preserve the history of complaint updates so that authorized users can understand how a complaint progressed from submission to resolution.

Complaint records shall be retained, archived, or deleted according to the institution's data-retention policy. Backups shall be protected and periodically tested for restoration.

9. System Constraints and Limitations

The initial version of the Smart Student Grievance and Complaint Management System has the following constraints and limitations.

1. **Network dependency:** Remote users require a working network connection and an accessible server.
2. **Database dependency:** The application depends on MongoDB availability for storing and retrieving persistent records.
3. **Authentication dependency:** Access to protected functions depends on correct authentication and authorization configuration.
4. **Notification limitations:** Email or SMS notifications will not work unless the corresponding external service is configured.
5. **Attachment limitations:** File uploads, if enabled, require suitable storage, file validation, and access controls.
6. **Manual complaint review:** Complaint verification and resolution depend on authorized staff unless automation is introduced.
7. **Limited automation:** The initial version may not automatically classify complaints, detect duplicate grievances, or predict resolution time.
8. **Scalability limitations:** Performance depends on server resources, database indexes, application design, and concurrent usage.
9. **Data accuracy:** Reports and dashboard statistics depend on accurate and complete records.
10. **Privacy and policy constraints:** Collection, access, retention, and deletion of student information must follow applicable institutional rules and legal requirements.

These limitations define the boundaries of the initial release and help identify areas for future development.

10. Future Enhancements

The system can be improved through additional features that enhance accessibility, automation, and complaint resolution.

10.1 AI-Based Complaint Classification

Artificial intelligence can classify complaints into categories such as hostel, academic, transport, infrastructure, and sanitation. This can reduce manual categorization effort and help route complaints to the appropriate department.

10.2 Automatic Priority Detection

A rule-based or machine-learning model could identify complaints requiring urgent attention based on their content and predefined institutional policies. Human review should remain available for important decisions.

10.3 Email and SMS Notifications

Students could receive notifications when a complaint is submitted, assigned, updated, or resolved. Reminder notifications could also be sent when complaints remain pending beyond specified time limits.

10.4 Mobile Application

A dedicated Android or iOS application could provide easier access to complaint submission, status tracking, and notifications.

10.5 Complaint Escalation

Complaints that remain unresolved beyond an approved deadline could be escalated to a higher-level administrator. Escalation rules should be configurable by the institution.

10.6 Analytics and Reporting

The administrator dashboard could include charts and reports showing:

- Total complaints by category.
- Pending and resolved complaint counts.
- Average resolution time.
- Department-wise complaint distribution.
- Monthly complaint trends.
- Complaints exceeding their target resolution time.

10.7 Anonymous Grievance Submission

The institution could introduce an anonymous reporting option for appropriate categories. Such a feature would need a carefully designed privacy policy, abuse-prevention measures, and a clear explanation of its limitations.

10.8 Feedback and Satisfaction Ratings

Students could provide feedback after resolution. The institution could use these ratings to evaluate service quality and identify recurring problems.

10.9 Duplicate Complaint Detection

The system could identify potentially similar complaints and alert administrators, helping them recognize recurring issues and common causes.

10.10 Advanced Security and Audit Features

Future versions could introduce multi-factor authentication, stronger audit reporting, automated backup monitoring, and security alerts for suspicious activities.

11. Conclusion

The Smart Student Grievance and Complaint Management System provides a structured digital approach to handling student complaints within an educational institution. By replacing or supplementing manual complaint procedures with a centralized web application, the system aims to improve accessibility, transparency, accountability, and efficiency.

The proposed system supports student registration, secure authentication, complaint submission, complaint tracking, administrator dashboards, status management, and user administration. Its architecture separates the user interface, application logic, and database, allowing the application to be maintained and expanded more easily.

The use of Python, Flask, HTML, CSS, JavaScript, and MongoDB provides a practical foundation for implementation. Functional and non-functional requirements define the expected behavior, performance, usability, security, reliability, and maintainability of the application.

The initial version establishes the foundation for future enhancements, including AI-based complaint classification, automated notifications, escalation mechanisms, analytics, and mobile access. With appropriate testing, security controls, and institutional policies, the proposed system can help create a more organized, responsive, and student-friendly grievance management process.

12. References

1. IEEE. IEEE Std 29148: Systems and Software Engineering — Life Cycle Processes — Requirements Engineering. IEEE Standards Association.
2. International Organization for Standardization (ISO). ISO/IEC 25010: Systems and Software Quality Models. ISO.
3. Python Software Foundation. Python Documentation. Available at: https://docs.python.org/3/
4. Pallets Projects. Flask Documentation. Available at: https://flask.palletsprojects.com/
5. MongoDB, Inc. MongoDB Manual. Available at: https://www.mongodb.com/docs/manual/
6. Mozilla. MDN Web Docs — HTML, CSS, and JavaScript. Available at: https://developer.mozilla.org/
7. OWASP Foundation. OWASP Application Security Verification Standard. Available at: https://owasp.org/www-project-application-security-verification-standard/
8. Git. Git Documentation. Available at: https://git-scm.com/doc
9. Bootstrap. Bootstrap Documentation, if Bootstrap is used in the implementation. Available at: https://getbootstrap.com/docs/
10. Institution-specific grievance policies and student-handbook guidelines, where applicable.